



Facultatea
de Automatică
și Calculatoare
Universitatea Tehnică
din Cluj-Napoca



Fundamental Programming Techniques

Assignment 3 - Support Presentation (Part 1)

Computer Science Department, TUCN

Spring 2026

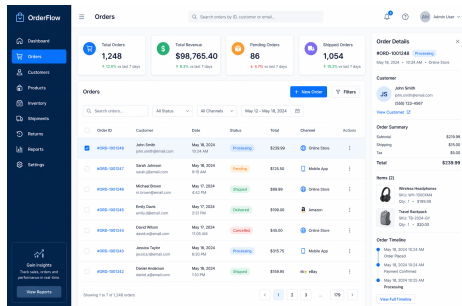
Problem and solution

PROBLEM: Managing the products, the clients and the orders for a warehouse using hand-written registries is difficult and time consuming.

SOLUTION: Order Management Application.

How to design and implement the solution?

1. Clearly state the main objective and the sub-objectives required to reach it.
2. Analyze the problem and define the functional and non-functional requirements.
3. Design the solution.
4. Implement the solution.
5. Test the solution.



Objectives

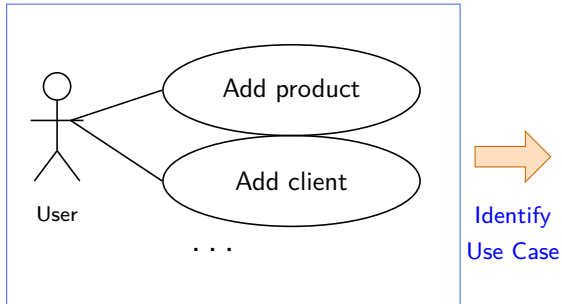
Main objective

- Design and implement an application for managing the client orders for a warehouse.

Sub-objectives

- Analyze the problem and identify requirements
- Design the orders management application
- Implement the orders management application
- Test the orders management application

Analysis - Use Case Definition



Fragment of the Use Case Diagram

What other use cases can you identify?

Can you identify use cases connected with *extend*, *include*, *generalize* relationships?

Use Case: add product
Primary Actor: employee
Main Success Scenario:

- The employee selects the option to add a new product.
- The application will display a form in which the product details should be inserted.
- The employee inserts the name of the product, its price and current stock.
- The employee clicks on the *Add* button.
- The application stores the product data in the database and displays an acknowledge message.

Alternative Sequence: Invalid values for product's data

- The user inserts a negative value for the stock of the product
- The application displays an error message and requests the user to insert a valid stock
- The scenario returns to step 3

Analysis - Requirements Definition

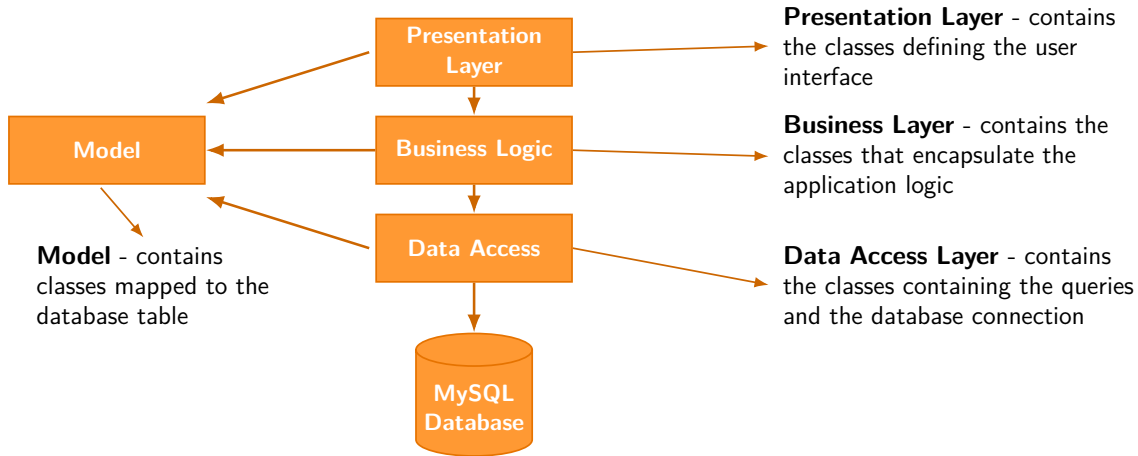
Functional Requirements:

- The application should allow an employee to add a new client.
- The application should allow an employee to add a new product.
- ...What other functional requirements can you define?

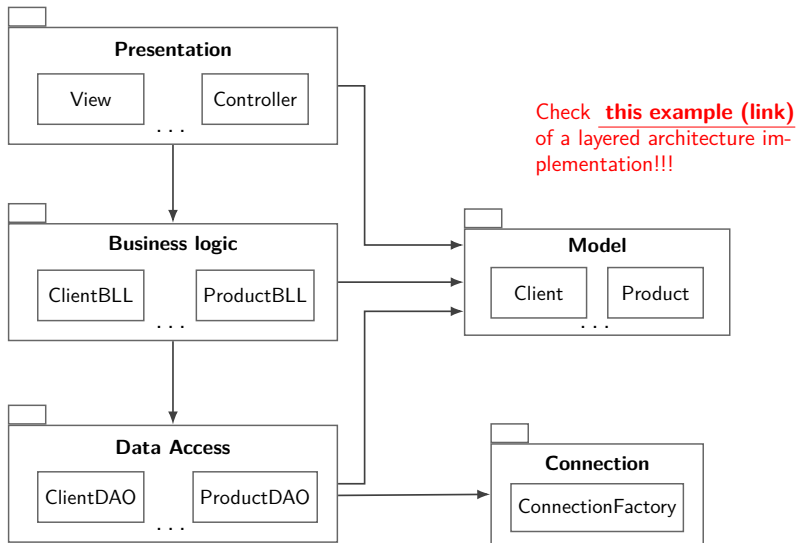
Non-Functional Requirements:

- The application should be intuitive and easy to use by the user.
- ...What other non-functional requirements can you define?

Design - Conceptual Architecture

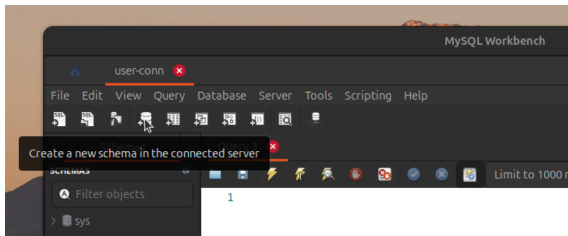


Design - Detailed Architecture

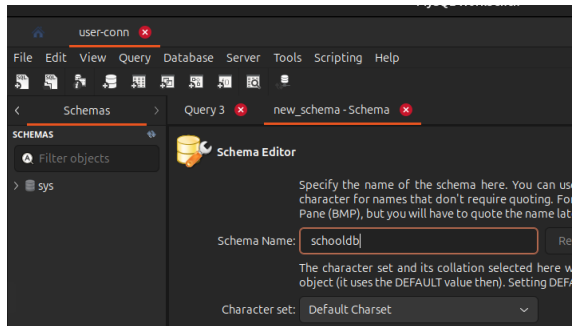


JDBC Basics - Prerequisites

- Install MySQL Server and MySQL Workbench (see [Lab Resources: section 3. Database Server](#))
- Create a database in MySQL Workbench — set *schooldb* as the name of schema



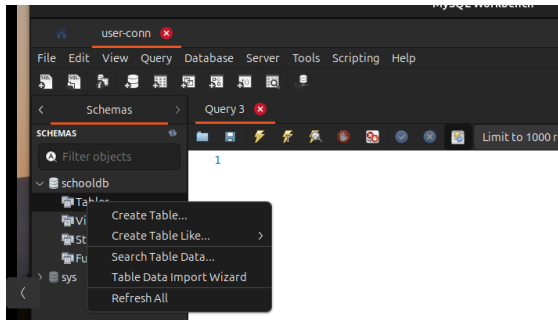
step 1



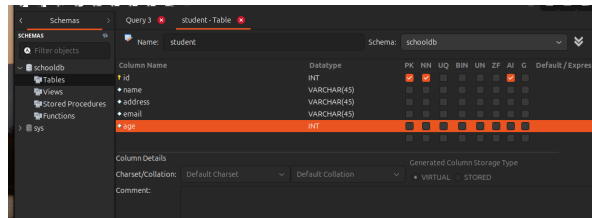
step 2

JDBC Basics - Prerequisites

- Create a table called *student*



step 3



step 4

JDBC Basics - Processing SQL Statements

Steps

- Establish a connection with the data source;
- Create a statement;
- Execute the query;
- Process the `ResultSet` object;
- Close the connection

JDBC Basics - Establishing a Connection

- This class contains the name of the driver(initialized through reflection), the database location (*DB_URL*), and the user and the password for accesing the *MySQL* server.
- The connection to the *DB* will be placed in a *Singleton** object.
- The class contains methods for creating a connection, getting an active connection and closing a Statement or a *ResultSet*

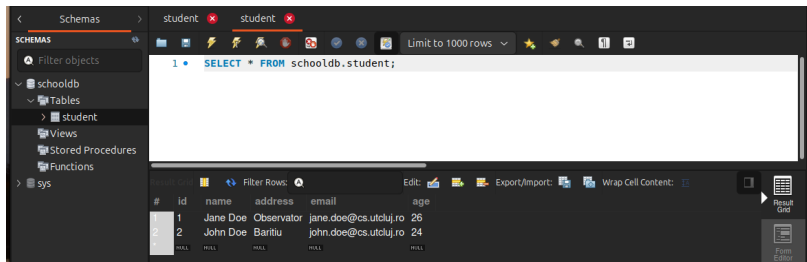
```
1 public class ConnectionFactory {
2     private static final Logger LOGGER = Logger.getLogger(
3         ConnectionFactory.class.getName())
4     private static final String DRIVER = "com.mysql.cj.jdbc.Driver";
5     private static final String DB_URL = "jdbc:mysql://localhost:3306/schooldb";
6     private static final String USER = "root";
7     private static final String PASS = "root";
8
9     private static ConnectionFactory singleInstance = new ConnectionFactory();
10
11     private ConnectionFactory() {
12         try{
13             Class.forName(DRIVER);
14         } catch(ClassNotFoundException e) {
15             e.printStackTrace();
16         }
17     }
18
19     private Connection createConnection() {...}
20
21     public static Connection getConnection() {...}
22
23     public static void close(Connection connection) {...}
24
25     public static void close(Statement statement) {...}
26
27     public static void close(ResultSet resultSet) {...}
28 }
```

Singleton Design Pattern: <https://refactoring.guru/design-patterns/singleton>

JDBC Basics - Table Mapping

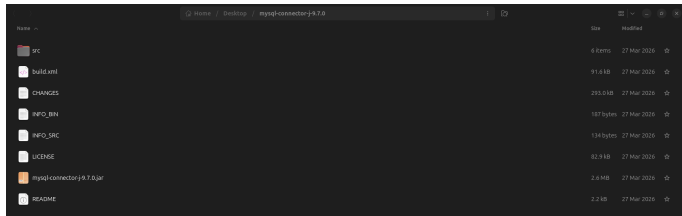
- In order to extract elements from the DB table, a special class (named entity) must be created.
- This class **MUST** have the fields exactly the same type as the columns from the corresponding table.
- The class must have also constructors, getters and setters.

```
1 public class Student {  
2     private int id;  
3     private String name;  
4     private String address;  
5     private String email;  
6     private int age;  
7 }
```



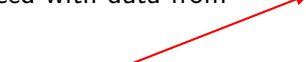
JDBC Basics - Dependencies

- In order for the Java application to interact with the *DB*, a special .jar library must be added to the application.
- It can be added either as an **external jar file dependency** or as a **maven dependency**, in case of a Maven project.



- The Java application uses this external library to communicate with the *MySQL* server.
- It sends queries to the server using Statements and it receives the results of the queries as ResultSet.
- Source for MySQL Connector file dependency: [source](#). Select Platform Independent at OS selection!

JDBC Basics - Creating and Executing Statement

- Define a string with the query  1 `private final static String findStatement =`
2 `"SELECT * FROM student WHERE id = ?";`
3
- Create a connection to the DB  4 `public static Student findById(int studentId) {`
5 `Student toReturn = null;`
6
7 `Connection dbConnection = ConnectionFactory.`
8 `getConnection();`
9 `PreparedStatement findStatement = null;`
10 `ResultSet rs = null;`
11 `try {`
- Initialize the query  12 `findStatement = dbConnection.`
13 `prepareStatement(findStatementString);`
14 `findStatement.setLong(1, studentId);`
15 `rs = findStatement.executeQuery();`
16 `} // rest of code`
17 `}`
- Add the parameters to the query
(the ? will be replaced with data from
the application) 
- Execute the query 

```
1 private final static String findStatement =
2 "SELECT * FROM student WHERE id = ?";
3
4 public static Student findById(int studentId) {
5     Student toReturn = null;
6
7     Connection dbConnection = ConnectionFactory.
8         getConnection();
9     PreparedStatement findStatement = null;
10    ResultSet rs = null;
11    try {
12        findStatement = dbConnection.
13            prepareStatement(findStatementString);
14        findStatement.setLong(1, studentId);
15        rs = findStatement.executeQuery();
16    }
17    // rest of code
18 }
```

JDBC Basics - Process the ResultSet object

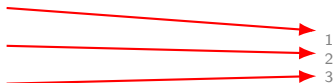
- The results of the query execution are stored in a result set
- Each element of the result set corresponds to a row from the table
- The result set can be iterated
- The properties(values) from the columns can be extracted if the column name is known

1
2
3
4
5
6
7
8

```
rs = findStatement.executeQuery();  
if(rs.next()) {  
    String name = rs.getString("name");  
    String address = rs.getString("address");  
    String email = rs.getString("email");  
    int age = rs.getInt("age");  
}  
}
```

JDBC Basics - Closing the connection

- After each operation the connection must be closed
- The result set
- The statement
- The connection



1
2
3

```
ConnectionFactory.close(rs);  
ConnectionFactory.close(findStatement);  
ConnectionFactory.close(dbConnection);
```